SESSION 4 OF 8

AI-Powered Testing and QA Automation

DAY 2 — AI Across the SDLC

Duration: 90 Minutes | QA Engineers + Developers + Analysts

Anurag Kumar

QUICK RECAP

Session 3 Takeaways + Where We Are

What we learned in Session 3:

- Raw requirements are ambiguous — Copilot finds the gaps
- One requirement → 15+ user stories with acceptance criteria
- AI-assisted product thinking works for all roles

NOW

Session 4

Those acceptance criteria from Session 3? Now we turn them into tests.

Session 3 defined what to test. Session 4 generates how to test it.

SESSION GOAL

Massively accelerate QA workflows with AI-generated tests

Unit tests, integration tests, test data, test strategy — all from prompts.

AI-generated tests are a starting point, not a finish line. Always review for correctness and business relevance.

WHY THIS SESSION MATTERS

Different Roles, Same Testing Mission

QA ENGINEERS

This is your session. Generate test plans, scenarios, and test data from acceptance criteria — without writing a line of Java.

DEVELOPERS

You'll generate JUnit tests, mocks, and integration tests for the Banking API. Ship code with confidence.

FUNCTIONAL ANALYSTS

See how your acceptance criteria from Session 3 translate directly into executable tests. The loop closes here.

SESSION AGENDA

90 Minutes | 20 min concept + 60 min hands-on + 10 min discussion

01 **AI-Generated Unit Tests**
JUnit 5, Mockito, parameterized tests from existing code

02 **Integration Test Generation**
MockMvc, @SpringBootTest, full request/response cycle

03 **Test Scenarios & Test Data**
Test matrices, synthetic fintech data, data factories

04 **Negative & Boundary Testing**
Zero amounts, max values, concurrency, security patterns

05 **TDD with AI**
Write tests first, Copilot generates the implementation

THE TESTING GAP

Our Codebase is Undertested

Current State: TransferServiceTest.java

```
@SpringBootTest
class TransferServiceTest {

  @Test void successfulTransfer() { ... }
  @Test void insufficientFunds() { ... }
  @Test void accountNotFound() { ... }
  @Test void invalidAmount() { ... }

  // That's it. 4 tests.
  // No mocks. No boundaries.
  // No parameterized tests.
  // No integration tests.
}
```

What We Need

- Boundary tests ($0, $0.01, $50,000, $50,001)
- Negative tests (frozen account, self-transfer)
- Mockito mocks for FraudCheckService, AuditService
- Parameterized tests for all 9 validation rules
- MockMvc integration tests for REST endpoints
- Test data factories for realistic scenarios
- Comprehensive test scenarios for QA

We'll go from 4 tests to 30+ in the next 60 minutes.

UNIT TESTS

JUnit 5 + Mockito from Existing Code

Generate from Code

Select a service method → Ask Copilot → Get JUnit 5 tests with assertions covering success, failure, and edge cases

Mock Dependencies

Copilot generates @Mock for repositories, @InjectMocks for services, and when/thenReturn for each dependency

Parameterized Tests

@ParameterizedTest with @ValueSource or @MethodSource for testing multiple inputs: $0, $0.01, $1.00, $50,000, $50,001

Validation Rule Tests

One test per validation rule in TransferService — all 9 rules covered with specific assertions and error messages

INTEGRATION TESTS

MockMvc: Testing the Full HTTP Cycle

Integration tests hit the real controller, serialization, validation, and error handling:

```
@SpringBootTest
@AutoConfigureMockMvc
class TransferControllerTest {

  @Autowired MockMvc mockMvc;

  @Test void validTransfer_returns200() throws Exception {
    mockMvc.perform(post("/api/transfers")
      .contentType(MediaType.APPLICATION_JSON)
      .content(validTransferJson))
      .andExpect(status().isOk())
      .andExpect(jsonPath("$.status").value("COMPLETED"));
  }
```

Copilot generates the full test class including JSON payloads, assertions, and error scenarios.

TEST DATA & SCENARIOS

Generating What to Test and With What Data

Test Scenario Matrix

- Functional: all 9 validation rules
- Boundary: min/max amounts, exactly-at-limit
- Negative: invalid input, missing fields, null values
- Security: SQL injection in description, XSS in fields
- Performance: high-frequency transfers to same account
- Compliance: AML threshold, OFAC screening

Synthetic Test Data

- Realistic account holders with varied balances
- Transaction histories spanning 12 months
- Edge case accounts: zero balance, max balance
- Test data builders with fluent API
- No real customer data — all synthetic
- Copilot generates the factory classes

NEGATIVE & BOUNDARY

Testing What Should Fail

The most important tests are the ones that verify failure. Copilot excels at generating these:

Category	Test Values	Expected Result
Amount boundaries	$0, -$100, $0.001, $0.99, $50,000.01, null	400 Bad Request with specific error message
Account state	FROZEN, CLOSED, non-existent, self-transfer	400/404 with account-specific error
Balance edge	Exact balance, $0.01 over balance, concurrent drain	Success at exact, failure at $0.01 over
Input validation	null fields, empty strings, 256-char description	400 with field-specific validation error
Fraud triggers	$10,001 transfer, "urgent" in description	Success but flagged in audit with risk score

TDD WITH AI

Write the Test First, Let AI Write the Code

The classic Red-Green-Refactor cycle, supercharged with Copilot:

1

RED

Write a failing test that describes the behavior you want

→

2

GREEN

Ask Copilot: “Make this test pass.” It generates the implementation

→

3

REFACTOR

Ask Copilot to clean up both the test and the implementation

TDD + AI = You define the behavior, Copilot delivers the implementation.

USE CASES

Real-World Testing Scenarios

DEVELOPER

20+ Tests in 10 Minutes

Generate comprehensive JUnit tests for TransferService covering all 9 validation rules, success, and edge cases.

QA ENGINEER

Test Plan Without Code

Generate a structured test scenario document covering functional, security, performance, and compliance tests.

DEVELOPER

Parameterized Boundaries

One parameterized test class validates all amount boundaries: $0, $0.01, $1, $50K, $50K.01, null, negative.

QA ENGINEER

Synthetic Test Data

Generate realistic but fake banking data: account holders, balances, 12-month transaction histories. No PII.

HANDS-ON TIME

60 Minutes of Test Generation with AI

Goal: Go from 4 tests to 30+ by the end of this session

EXERCISE 1

Generate JUnit Tests for TransferService

What to do:

Open TransferServiceTest.java (currently has 4 basic tests). Select the TransferService class, open Copilot Chat in Ask mode, and ask for comprehensive tests.

Try this prompt:

```
Generate comprehensive JUnit 5 tests for
TransferService. Include:
- Success case with valid transfer
- All 9 validation rule failures
- Boundary conditions ($0, $1, $50,000, $50,001)
- Fraud check flagging scenarios
Use Mockito for repository and service mocks.
```

Pro Tip: After generating, run mvn test. How many pass on the first try? That tells you about Copilot's accuracy.

EXERCISE 2

Parameterized Tests + Mockito Setup

What to do:

Ask Copilot to generate a parameterized test for amount validation and a proper Mockito setup with @Mock and @InjectMocks for all dependencies.

Try this prompt:

```
@ParameterizedTest with @ValueSource for validateAmount: test with these values:
$0, -$100, $0.001, $0.99, $1.00, $25000, $50000, $50000.01, null

Also generate @BeforeEach setup with
@Mock AccountRepository, TransactionRepository, FraudCheckService, AuditService
@InjectMocks TransferService
```

Pro Tip: Check if Copilot correctly distinguishes which values should pass vs fail. The boundary at $1.00 and $50,000 is where mistakes happen.

EXERCISE 3

MockMvc Integration Tests

What to do:

Generate integration tests for POST /api/transfers using MockMvc. Test valid requests, invalid payloads, missing fields, and error responses.

Try this prompt:

```
Generate MockMvc integration tests for
POST /api/transfers. Include tests for:
- Valid transfer → 200 with TransferResponse
- Missing fromAccountId → 400
- Amount exceeds $50,000 → 400
- Non-existent account → 404
- Insufficient funds → 400
Use @SpringBootTest + @AutoConfigureMockMvc
```

Pro Tip: Integration tests catch serialization bugs, validation annotation errors, and error handler issues that unit tests miss.

EXERCISE 4

QA: Generate Test Scenario Document

What to do:

This exercise is especially for QA engineers. No Java code needed. Ask Copilot to generate a comprehensive test plan as a document.

Try this prompt:

```
Generate a comprehensive test scenario document for the fund transfer feature.
Organize into:
1. Functional tests (happy path + all error cases)
2. Boundary tests (amounts, string lengths)
3. Security tests (injection, IDOR, auth)
4. Performance tests (concurrent transfers)
5. Compliance tests (AML, KYC thresholds)
Format as a table: Scenario | Steps | Expected | Priority
```

Pro Tip: This output can go directly into Jira or your test management tool. Ask: would you add or remove any scenarios?

EXERCISE 5

TDD: Write Test First, AI Implements

What to do:

Write a failing test for a new feature: daily transfer limit of $100,000 per account. Don’t implement the feature — just write the test. Then ask Copilot to make it pass.

Try this prompt:

```
Step 1 – Write this test (it will fail):
"Test that a transfer is rejected when the account has already transferred $95,000
today and attempts to transfer $10,000 more."

Step 2 – Ask Copilot:
"Make this test pass. Implement the daily transfer limit check in TransferService."
```

Pro Tip: This is the TDD + AI workflow: you define behavior through tests, Copilot delivers the implementation. Review what it generates!

PROMPT REFERENCE

Session 4 — All Key Prompts

1. Generate JUnit 5 tests for TransferService: success, all 9 validation failures, boundaries. Use Mockito mocks.

2. Create @ParameterizedTest for amount validation with $0, -$100, $0.99, $1.00, $50,000, $50,000.01, null.

3. Generate MockMvc integration tests for POST /api/transfers: valid, missing fields, over limit, not found, insufficient.

4. Generate test scenario document for fund transfers: functional, boundary, security, performance, compliance as table.

5. Write a failing test for daily $100K transfer limit. Then: make this test pass by implementing the limit check.

AI as QA Co-Engineer

AI-generated tests are a starting point, not a finish line.
Always review for correctness, completeness, and business relevance.

Reflection

- How many of the generated tests passed on the first run?
- Which tests did Copilot miss that you would have written?
- How would TDD + AI change your team's development workflow?

UP NEXT — DAY 3

Session 5

Copilot CLI &
Developer Workflows